

Analyse de complexité

rouanehm@gmail.com

Complexité des algorithmes

- L'objet de la théorie de la complexité est de mesurer les ressources principalement le temps et la mémoire utilisées par un programme ou nécessaires à la résolution d'un problème.
- Le but est de :
 - Évaluer l'efficacité d'une méthode de calcul
 - Comparer les méthodes de calcul entre elles indépendamment de l'environnement d'exécution (Machine, Compilateur, etc.)
- La théorie de la complexité permet de formaliser proprement la notion d'algorithme, en complète généralité, en faisant abstraction du langage utilisé

Structures algorithmiques

- Structures de contrôle
 - Séquence
 - Embranchement (ou sélection)
 - La boucle (ou itération)
- Structures de données : support des informations
 - Variables
 - Constantes
 - Tableaux
 - Objets
 - Structures récursives (listes, arbres, graphes)

Notion de ressource élémentaire

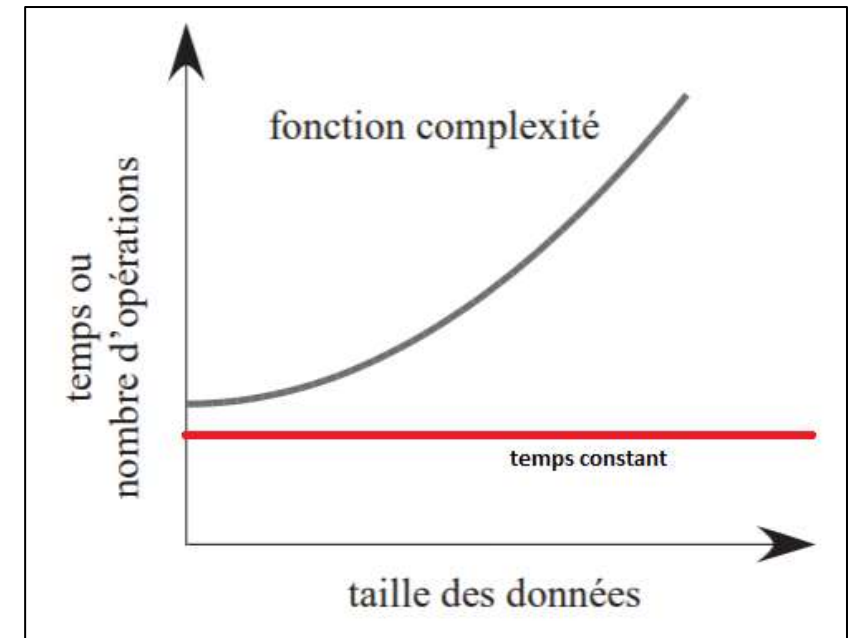
- On mesure toujours l'efficacité, c'est-à-dire la complexité, d'un algorithme en termes d'une mesure élémentaire à valeur entière.
- Cela peut être :
 - La taille de la mémoire utilisée
 - Le nombre d'affectations effectuées
 - Le nombre de comparaisons effectuées
 - Ou toute autre mesure.

Nombre d'opérations

- Le nombre d'opérations élémentaires est calculé en fonction de :
 - La taille des données (par ex. le nombre d'éléments à trier)
 - La nature des données (provoquant par ex. la sortie d'une boucle)
- Notations :
 - n : taille des données,
 - $T(n)$: nombre d'opérations élémentaires
 - noté aussi $C(n)$: le cout de traitement de données de taille n .
- Plusieurs catégories de complexité:
 - Au pire cas (par défaut quand on aborde la complexité)
 - Au cas moyen (Quand on connaît la distribution des données d'entrée)
 - Au meilleur cas

Ordre constant

- Le temps de calcul ne varie pas avec la taille des données en entrée.
 - $C(n)=C(n+1)$: que l'on traite des données de taille n ou de taille supérieure, le temps de calcul est le même.
- Noté $O(1)$
- Exemples :
 - L'accès à `tableau[3]` est une opération en $O(1)$, car le temps nécessaire pour la réaliser ne dépend pas de la longueur du tableau. Il s'agit d'une opération simple et directe, peu importe que le tableau contienne 5 items ou 5 millions d'items
 - L'ajout ou la suppression d'un élément à la fin d'une collection



Calcul du nombre d'opérations

- La séquence :
 - On prend la somme des coûts
- Embranchement :
 - On prend le max des coûts dans les branches
- Boucle
 - On prend la somme des coûts des passages successifs
- Fonction récursive
 - Équation récursive

```

1  si (n > 1) alors
2      FUNCTIONRECURSIVE(n/2),      coût T(n/2)
3      Traitement(n),              coût C(n)
4      FUNCTIONRECURSIVE(n/2),      coût T(n/2)

```

$$T(n) = 1 + 2 \times T(n/2) + C(n)$$

Traitement1	$T_1(n)$	} $T(n) = T_1(n) + T_2(n)$
Traitement2	$T_2(n)$	

si < condition > alors	$T_c(n)$	} $T_c(n) + \max(T_1(n), T_2(n))$
Traitement1	$T_1(n)$	
sinon		
Traitement2	$T_2(n)$	

tant que < condition > faire	$C_i(n)$	} $\sum_{i=1}^k (C_i(n) + T_i(n)) + C_{k+1}(n)$
Traitement	$T_i(n)$	
fin faire		

Complexité asymptotique et ordre de grandeur

Ou quand n tend vers l'infini

- On s'intéresse le plus souvent à l'ordre de grandeur (l'asymptotique) des complexités quand la taille n des entrées devient très grande.
 - Exemple : linéaire, quadratique, exponentiel, etc.

$$T(n) = n^3 + 2n^2 + 4n + 2 = \mathcal{O}(n^3)$$

$$T(n) = n \log n + 12n + 2 = \mathcal{O}(n \log n)$$

$$T(n) = 2n^{10} + n^7 + 12n^4 + \frac{2^n}{100} = \mathcal{O}(2^n)$$

Classes de complexité usuelles

$\mathcal{O}(\log n)$

logarithmique

$\mathcal{O}(n)$

linéaire

$\mathcal{O}(n^2)$

quadratique, polynomial

$\mathcal{O}(n^3)$

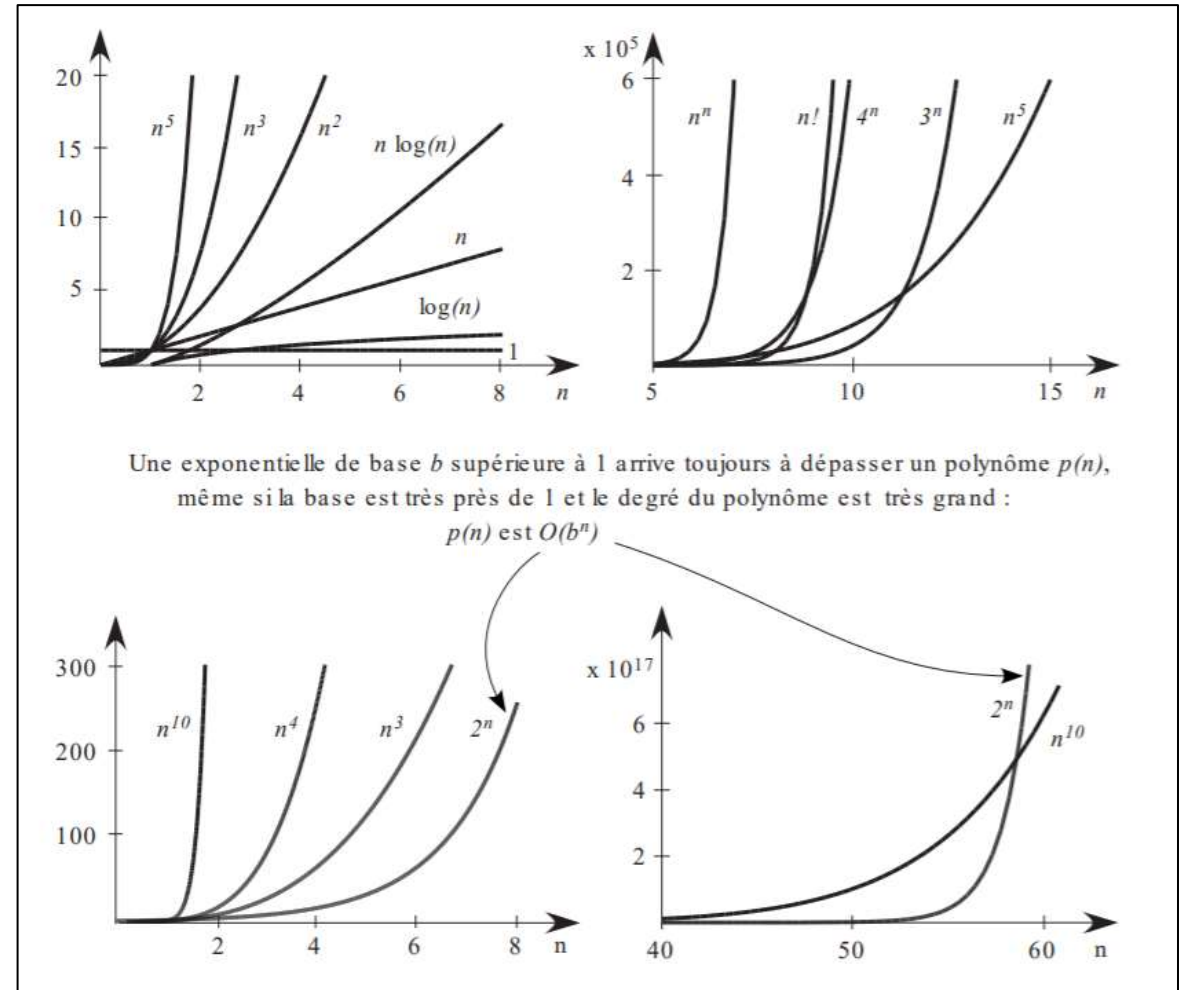
cubique, polynomial

$\mathcal{O}(2^n)$

exponentiel (problèmes très difficiles)

$1, \log(n), n, n \log(n), n^2, n^3, n^4, \dots, 2^n, 3^n, 4^n, \dots, n!, n^n$

Classement des ordres de grandeurs



Ordre de grandeur et temps d'exécution

- Le temps d'exécution est proportionnel à l'ordre de grandeur du nombre d'instructions élémentaires.
- Pour une machine qui effectue 1 million de traitements par seconde.

Complexité	n	$n \log_2 n$	n^2	n^3	$(\frac{3}{2})^n$	2^n	$n!$
$n = 10$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	4 s
$n = 30$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	18 min	10^{25} ans
$n = 50$	< 1 s	< 1 s	< 1 s	< 1 s	11 min	36 ans	∞
$n = 100$	< 1 s	< 1 s	< 1 s	1s	12,9 ans	10^{17} ans	∞
$n = 1000$	< 1 s	< 1 s	1s	18 min	∞	∞	∞
$n = 10000$	< 1 s	< 1 s	2 min	12 jours	∞	∞	∞
$n = 100000$	< 1 s	2 s	3 heures	32 ans	∞	∞	∞
$n = 1000000$	1s	20s	12 jours	31 710 ans	∞	∞	∞

Calcul de complexité

Exemples

- Permutation dans un tableau

fonction PERMUTATION (S, i, j)

1	$tmp := S[i],$	coût c_1
2	$S[i] := S[j],$	coût c_2
3	$S[j] := tmp,$	coût c_3
4	renvoyer S	coût c_4

$$T(n) = c_1 + c_2 + c_3 + c_4 = \mathcal{O}(1)$$

- Recherche séquentielle

fonction RECHERCHE_SEQUENTIELLE($x, S[1, \dots, n]$)

1	$i := 1,$	(c_1)
2	tant que $((i < n) \text{ et } (S[i] \neq x))$ faire	(c_2)
3	$i := i + 1,$	(c_3)
4	renvoyer $(S[i] = x)$	(c_4)

$$T(n) = c_1 + c_2 + \sum_{i=1}^n (c_3 + c_2) + c_4 = \mathcal{O}(n)$$

Calcul de complexité

Exemples

```
fonction TRI_A_BULLES ( $S[1, \dots, n]$ )  
1  pour  $i := n$  à 2 faire  
2      pour  $j := 1$  à  $i - 1$  faire  
3          si ( $S[j] > S[j + 1]$ ) alors  
4              PERMUTER( $S, j, j + 1$ ),
```

$$\begin{aligned} T(n) &= (1 + C_{perm}) \times \sum_{i=1}^{n-1} i \\ &= (1 + C_{perm}) \times \frac{n \times (n - 1)}{2} = \mathcal{O}(n^2) \end{aligned}$$

Calcul de complexité

Exemples

- Si on ne sait rien sur les données, on considère la vision pessimiste des choses : cela revient à évaluer la complexité dans le pire des cas
- En pratique, le pire cas est rarement atteint

```
def find_max(numbers):  
    if len(numbers) == 0:  
        return None  
  
    max_value = numbers[0]  
  
    for i in range(1, len(numbers)):  
        if numbers[i] > max_value:  
            max_value = numbers[i]  
  
    return max_value
```

Exemple. Considérons le problème du calcul du maximum : on se donne en entrée une liste d'entiers naturels $e_1, e_2, \dots, e_n$, avec $n \geq 1$, et on cherche à déterminer en sortie $M = \max_{1 \leq i \leq n} e_i$, c'est-à-dire le plus grand de ces entiers. Si l'entrée est rangée dans un tableau, la méthode Java suivante résout le problème.

```
static int max(int[] T) {  
    int M = T[0];  
    for (int i = 1; i < T.length; i++)  
        if (M < T[i]) M = T[i];  
    return M;  
}
```

Ordre de
grandeurs de
certaines
tâches
usuelles

Complexité	Tâche
$O(1)$	Accès direct à un élément
$O(\log n)$	Divisions successives par deux d'un ensemble
$O(n)$	Parcours d'un ensemble
$O(n \log n)$	Divisions successives par deux et parcours de toutes les parties
$O(n^2)$	Parcours d'une matrice carrée de taille n
$O(2^n)$	Génération des parties d'un ensemble
$O(n!)$	Génération des permutations d'un ensemble

Que faire fasse à un problème intraitable?

- Un problème NP-complet :
 - NP : Non-deterministic Polynomial time
 - très longs à résoudre
 - EX. « Existe-t-il un trajet de longueur ≤ 500 km qui visite toutes les villes une fois ? »
- Une question à un million 1 000 000 \$
 - Le fait qu'à ce jour aucun des problèmes NP-complets n'ait été résolu en temps polynomial nous porte à croire que cela est impossible. Mais l'impossibilité de les résoudre en temps polynomial n'a pas été démontrée. Cette question ouverte constitue d'ailleurs un des 7 problèmes du millénaire pour chacun desquels un million de dollars est offert en récompense par le Clay Mathematics Institute of Cambridge.